

Diapositivas

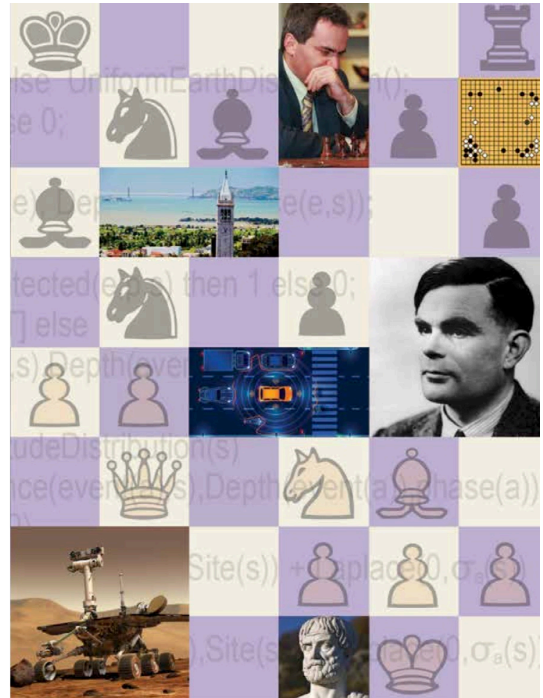
INTELIGENCIA ARTIFICIAL
SISTEMAS INTELIGENTES

CONSTRUCCIÓN DE SISTEMAS RAG CON LLAMAINDEX

MG. JUAN MANUEL FERNÁNDEZ



Laboratorio de Ciencia de Datos &
Inteligencia Artificial (LICDIA)



OUTLINE DE LA PRESENTACIÓN

- ★ ¿Qué es RAG?
- ★ Presentación de LlamaIndex
 - Componentes principales de LlamaIndex
 - Ingesta de datos: loaders y parsers
 - Indexación y almacenamiento vectorial
 - Recuperación: retrievers y queries
 - Integración con LLMs y generación aumentada (RAG)
- ★ Referencias

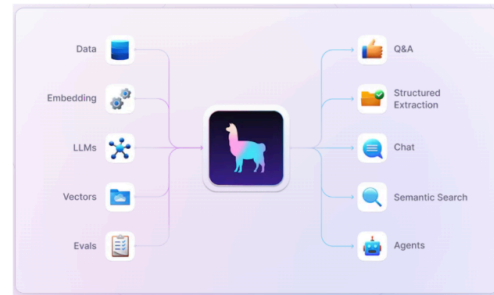


FRAMEWORKS PARA TRABAJAR CON LLM: LLAMAINDEX

LlamaIndex es un framework para crear aplicaciones LLM aumentadas por contexto. El aumento de contexto se refiere a cualquier solución que aplique un LLM a datos privados o específicos de un dominio.

Algunos casos de uso populares incluyen los siguientes:

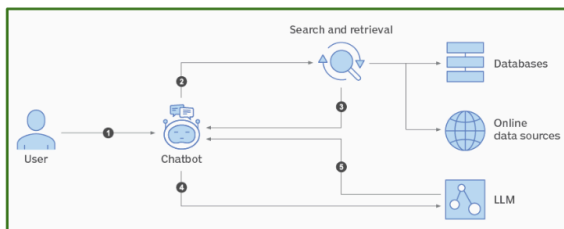
- ★ Sistemas de Retrieval-Augmented Generation (RAG) para recuperar información contextualmente relevante.
- ★ Chatbots con capacidad de consulta sobre bases documentales.
- ★ Extracción estructurada y comprensión de documentos.



LlamaIndex posee herramientas (para Python y Typescript) le para ingerir/procesar estos datos e implementar flujos de trabajo de consultas complejas que combinan el acceso a los datos con interacciones con los LLM.

¿QUÉ ES RAG?: UN CONCEPTO INTRODUCTORIO

- ★ El conocimiento de los LLM se limita a los datos públicos hasta un momento específico en el que fueron entrenados.
- ★ La Recuperación Aumentada de Información (RAG) permite superar esta limitación incorporando información externa (actual, privada o específica) a partir de sistemas de búsqueda.
- ★ Este enfoque marca un cambio respecto al paradigma tradicional, ya que las respuestas del modelo no se generan exclusivamente con base en lo aprendido durante el entrenamiento, sino con información contextual recuperada al momento de la consulta.

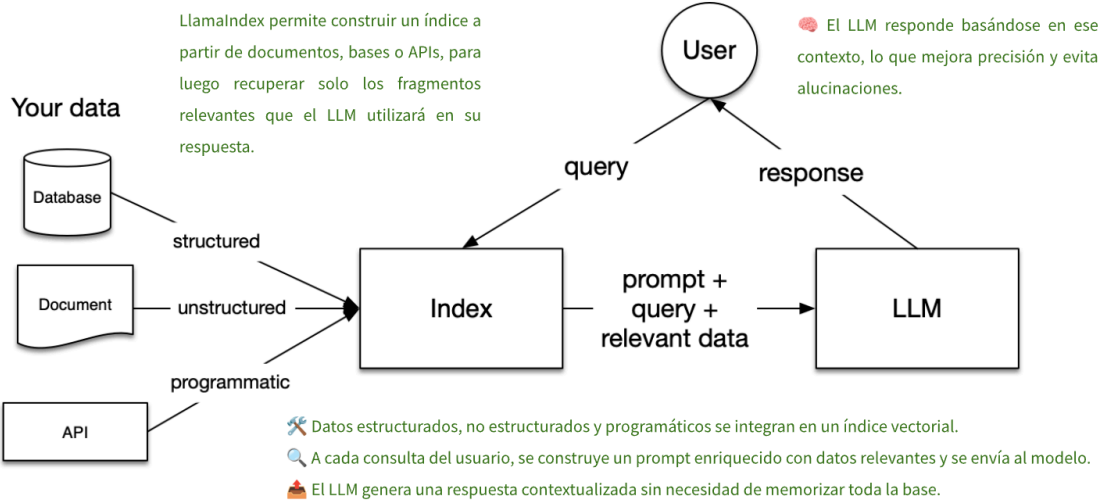


#1 El usuario realiza una consulta al Sistema.

#2 y #3 El sistema busca y recupera fragmentos relevantes de información mediante técnicas de recuperación (IR), desde fuentes internas o externas.

#4 y #5 El modelo LLM procesa esos fragmentos para generar una respuesta contextualizada, relevante y coherente.

APLICACIÓN RAG CON LLAMAINDEX



ETAPAS DEL PIPELINE RAG EN LLAMAINDEX



Loading (Documentos y nodos)

Un documento es cualquier fuente de datos (PDF, web, DB). Se divide en nodos, que son las unidades mínimas de información.

Connectors (también llamados Readers) permiten cargar información desde múltiples formatos.

Indexing (embeddings e índices)

LlamaIndex convierte los nodos en embeddings vectoriales y los organiza en un índice especializado (vector store) con metadatos asociados. Esto permite búsquedas semánticas eficientes sobre los datos.

Storing (Vector stores persistentes)

Los embeddings se almacenan en bases de datos vectoriales como Chroma, FAISS o Weaviate. Se preservan los metadatos, relaciones y estructura de los nodos para consultas futuras.

Querying (Retrieval + Reranking + Synthesis)

- Retrievers recuperan los nodos relevantes.
- Routers permiten elegir entre múltiples estrategias o índices.
- Postprocessors aplican filtros, ranking o transformación a los nodos.
- Synthesizers generan una respuesta del LLM combinando query + contexto.

Evaluating (Medición del desempeño)

Se pueden evaluar resultados por precisión, cobertura y relevancia.

LlamaIndex también permite hacer tracing de qué nodos y chunks fueron usados.



LLAMAINDEX: COMPONENTES PRINCIPALES

Componente	Rol
 Loaders	Cargan datos desde múltiples fuentes (PDF, CSV, web, etc.)
 Indexes	Estructuras para representar la información (vector, keyword, tree, list)
 Retrievers	Mecanismos para buscar información relevante en los índices
 Query Engines	Ejecutan consultas RAG, estructuran la interacción con el LLM
 Integraciones	Permiten combinar con LangChain, OpenAI, HuggingFace, etc.



1. READER: INGESTA DE DATOS DESDE MÚLTIPLES FUENTES

El componente Reader permite importar información desde diversas fuentes hacia el sistema LlamaIndex. Esta información puede ser estructurada (tablas, bases de datos), no estructurada (PDFs, texto, HTML) o generada de forma programática (APIs).

Los readers transforman estas fuentes en objetos Document (que contienen texto y metadatos), que luego serán divididos en Nodes para indexación.



- ★ Permiten conectar datos desde archivos, APIs o bases de datos con un solo comando.
- ★ Incluyen parsers automáticos para extraer el contenido relevante.
- ★ Existen muchos loaders específicos disponibles.

```
from llama_index.readers import SimpleDirectoryReader

documents = SimpleDirectoryReader("datos_pdfs/").load_data()
```



EJEMPLOS DE READERS COMUNES EN LLAMAINDEX

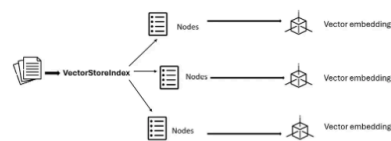
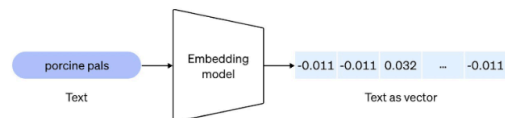
Reader	Fuente	Descripción
SimpleDirectoryReader	Carpeta local	Carga múltiples archivos desde una ruta. Ideal para PDFs, DOCX, TXT.
BeautifulSoupWebReader	Sitios web	Extrae contenido HTML desde URLs usando BeautifulSoup.
PDFReader	Archivos PDF	Lee directamente archivos PDF individuales o en lote.
PandasCSVReader	CSVs	Carga archivos CSV como DataFrames y los convierte en documentos.
NotionPageReader	Notion API	Permite cargar contenido estructurado desde Notion.
SQLDatabaseReader	SQL / SQLite	Ejecuta consultas sobre bases SQL y carga los resultados como documentos.



EMBEDDINGS: REPRESENTACIÓN SEMÁNTICA DEL CONTENIDO

Previo a indexar los documentos, se genera la representación mediante embeddings de los mismos.

Los embeddings son representaciones numéricas del significado de un texto y permiten comparar fragmentos por similitud conceptual en un espacio vectorial.



🌱 ¿Qué emebe?

- ★ Cada Node se convierte en un embedding.
- ★ El modelo de embedding debe definirse antes de construir el índice.

```
from llama_index.embeddings.openai import OpenAIEmbedding

embed_model = OpenAIEmbedding()
```





EJEMPLOS DE MODELOS DE EMBEDDING EN LLAMAÍNDX

Modelo	Proveedor	Descripción breve
OpenAIEmbedding	OpenAI	Usa modelos como <code>text-embedding-3-small</code> o <code>ada-002</code> . Requiere API key.
HuggingFaceEmbedding	Hugging Face	Compatible con modelos públicos o locales (ej: <code>all-MiniLM</code> , <code>sentence-BERT</code>).
CohereEmbedding	Cohere	Usa la API de Cohere. Bueno para clasificación y QA.
VoyageEmbedding	Voyage AI	Alta performance en benchmarks. Requiere clave.
JinaEmbedding	Jina AI	Modelo de código abierto, ideal para prototipos.
DefaultEmbedding	LlamaIndex	Modelo básico sin dependencias externas. Útil para pruebas.



EJEMPLOS DE MODELOS DE EMBEDDING EN



Tip

El modelo de embedding depende de:

- el idioma,
- si querés usar servicios externos o locales,
- y el trade-off entre performance y costo.

Modelo	Proveedor	Descripción b
OpenAIEmbedding	OpenAI	Usa modelos como <code>text-embedding-3-small</code>
HuggingFaceEmbedding	Hugging Face	Compatible con modelos públicos o locales (ej: <code>all-MiniLM</code> , <code>sentence-BERT</code>).
CohereEmbedding	Cohere	Usa la API de Cohere. Bueno para clasificación y QA.
VoyageEmbedding	Voyage AI	Alta performance en benchmarks. Requiere clave.
JinaEmbedding	Jina AI	Modelo de código abierto, ideal para prototipos.
DefaultE		



```
from llama_index.embeddings.huggingface import HuggingFaceEmbedding  
  
embed_model = HuggingFaceEmbedding(model_name="sentence-transformers/all-MiniLM-L6-v2")
```





INDEXING: ORGANIZACIÓN EFICIENTE PARA CONSULTAS

Una vez generados los embeddings, los Nodes se almacenan en una estructura llamada índice, junto con sus metadatos.

El índice permite realizar búsquedas eficientes, encontrar los fragmentos más relevantes y preparar el contexto para el LLM.

```
from llama_index import VectorStoreIndex

index = VectorStoreIndex.from_documents(
    documents,
    embed_model=embed_model,
    text_splitter=splitter # optional
)
```

Índice	Uso recomendado
VectorStoreIndex	Consultas semánticas (default en RAG).
KeywordTableIndex	Matching exacto por palabras clave.
TreeIndex	Resúmenes jerárquicos o navegación temática.
ListIndex	Flujos secuenciales como logs o scripts.

✚ El índice guarda los embeddings y metadatos de cada Node. Luego serán consultados por los Retrievers.



STORING: ALMACENAMIENTO PERSISTENTE DE EMBEDDINGS

Una vez generados los embeddings, estos pueden almacenarse en vector stores persistentes, que permiten:

- ★ Evitar recomputar embeddings en cada ejecución.
- ★ Compartir el índice entre distintos procesos.
- ★ Consultar eficientemente grandes volúmenes de información.



✚ ¿Qué se almacena?

- ★ Embeddings de cada nodo.
- ★ Metadatos: fuente, posición, título, etc.
- ★ Relación con el documento original.

```
from llama_index.vector_stores import ChromaVectorStore

vector_store = ChromaVectorStore(persist_dir="./chroma_store")
index = VectorStoreIndex.from_documents(
    documents,
    embed_model=embed_model,
    vector_store=vector_store
)
```





OTROS VECTOR STORES COMPATIBLES

Vector Store	Descripción
Chroma	Local, gratuito, fácil de usar. Ideal para prototipos y producción ligera.
FAISS	Rápido y optimizado para grandes volúmenes (Meta).
Weaviate	Vector DB con capacidades semánticas, API y consola web.
Pinecone	Vector DB como servicio en la nube, escalable.

A partir del almacenamiento de embeddings, luego es posible cargar un índice previamente almacenado usando su vector store, sin necesidad de re-embellido.



```
from llama_index.vector_stores import ChromaVectorStore
from llama_index import VectorStoreIndex

# 1. Reutilizar el vector store previamente guardado
vector_store = ChromaVectorStore(persist_dir="./chroma_store")

# 2. Cargar el índice directamente desde ese store
index = VectorStoreIndex.from_vector_store(vector_store)
```



QUERYING: RECUPERAR CONTEXTO DESDE EL ÍNDICE

En esta etapa, se utiliza un Retriever para recuperar los Nodes más relevantes del índice, dada una consulta del usuario.

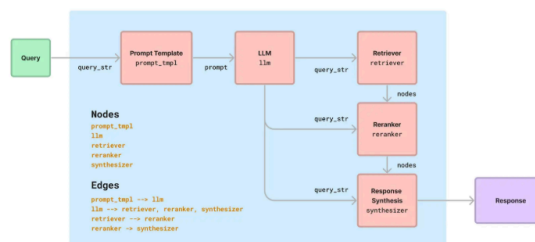
¿Qué hace un Retriever (básico)?

- ★ Convierte la consulta en un embedding.
- ★ Busca los Nodes más cercanos en el espacio vectorial del índice.
- ★ Devuelve un conjunto de fragmentos para usarlos como contexto.

```
query_engine = index.as_query_engine(similarity_top_k=5)
response = query_engine.query("¿Cuál es el objetivo del proyecto?")
```

similarity_top_k en LlamaIndex toma 3 nodos.

Query Pipeline



Este es el flujo mínimo para recuperar contexto y obtener una respuesta directa desde un índice en LlamaIndex.

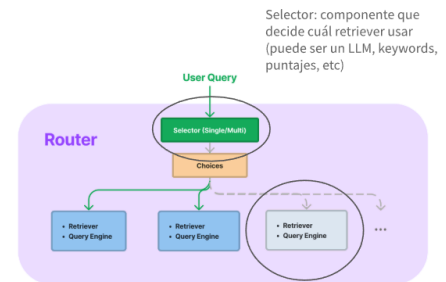


QUERYING: ROUTERRETRIEVER: ¿QUÉ ES Y CUÁNDO USARLO?

RouterRetriever permite elegir automáticamente entre múltiples retrievers (cada uno asociado a un índice) en función de la consulta del usuario.

¿Para qué sirve?

- ★ Cuando tienes varias fuentes de conocimiento (ej: FAQs, Documentos, Reglamentos, etc.)
- ★ El sistema elige dinámicamente el retriever más adecuado para responder la consulta.



Existen diferentes tipos de selectores



Selector	Descripción rápida
LLMSingleSelector	Usa un LLM para analizar la intención
KeywordIndexSelector	Elige según palabras clave
PydanticSelector	Elige según condiciones lógicas estructuradas



ROUTERRETRIEVER CON LLM VÍA API (EJ. OPENAI)

```
from llama_index.llms import OpenAI
from llama_index.selectors import LLMSingleSelector
from llama_index.retrievers import RouterRetriever

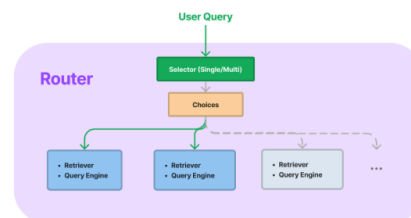
# Paso 1 - Configurar el modelo LLM (requiere tu API key)
llm = OpenAI(model="gpt-3.5-turbo")

# Paso 2 - Crear el selector
selector = LLMSingleSelector.from_llm(llm)

# Paso 3 - Definir los retrievers
retrievers = {
    "faq": index_faq.as_retriever(),
    "docs": index_docs.as_retriever()
}

# Paso 4 - Crear el RouterRetriever
router = RouterRetriever(retrievers=retrievers, selector=selector)

# Paso 5 - Ejecutar la consulta
nodes = router.retrieve("¿Dónde está la política de privacidad?")
```



Una forma común de usar RouterRetriever es con un selector que utiliza un LLM para interpretar la consulta del usuario y elegir el retriever más adecuado. El más usado es **LLMSingleSelector**.





ROUTERRETRIEVER CON LLM VÍA HUGGING FACE

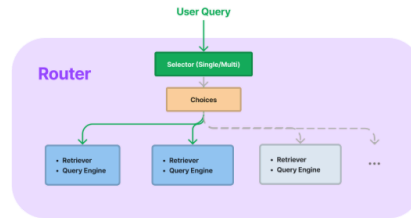
```
from llama_index.llms.huggingface import HuggingFaceInferenceAPI
from llama_index.selectors import LLMSingleSelector
from llama_index.retrievers import RouterRetriever

# Paso 1 - Configurar el LLM (requiere token de Hugging Face)
llm = HuggingFaceInferenceAPI(
    model_name="mistralai/Mistral-7B-Instruct-v0.1",
    token="HF_API_KEY"
)

# Paso 2 - Crear el selector
selector = LLMSingleSelector.from_llm(llm)

# Paso 3 - Definir Los retrievers
retrievers = {
    "faq": index_faq.as_retriever(),
    "docs": index_docs.as_retriever()
}

# Paso 4 - Crear el RouterRetriever
router = RouterRetriever(retrievers=retrievers, selector=selector)
```



También es posible utilizar RouterRetriever con un modelo local o alojado en Hugging Face, sin depender de servicios pagos ni conexión constante.



ROUTERRETRIEVER CON LLM LOCAL (OFFLINE)

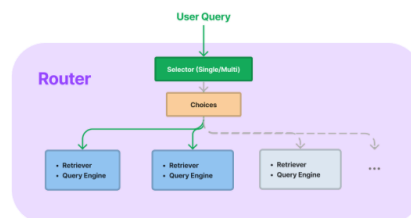
```
from llama_index.llms import HuggingFaceLLM
from llama_index.llms.huggingface import HuggingFaceLLMConfig
from llama_index.selectors import LLMSingleSelector
from llama_index.retrievers import RouterRetriever

# Paso 1 - Configurar el modelo Local
llm = HuggingFaceLLM(
    config=HuggingFaceLLMConfig(
        model_name="tiiuae/falcon-7b-instruct", # o cualquier modelo Local
        tokenizer_name="tiiuae/falcon-7b-instruct",
        context_window=2048,
        max_new_tokens=256,
        generate_kwargs={"temperature": 0.1, "do_sample": False},
    )
)

# Paso 2 - Crear el selector con el LLM Local
selector = LLMSingleSelector.from_llm(llm)

# Paso 3 - Definir Los retrievers
retrievers = {
    "faq": index_faq.as_retriever(),
    "docs": index_docs.as_retriever()
}

# Paso 4 - Crear el RouterRetriever
router = RouterRetriever(retrievers=retrievers, selector=selector)
```



LlamaIndex permite usar modelos descargados de Hugging Face en tu máquina, sin API externa. Esto es ideal para investigación, enseñanza offline o despliegues sin conexión.





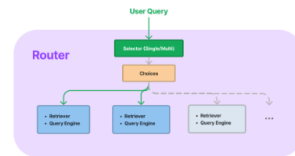
ROUTERRETRIEVER SIN LLM: SELECTOR BASADO EN PALABRAS CLAVE

```
from llama_index.selectors import KeywordIndexSelector

# Paso 1 - Crear el selector con palabras clave por índice
selector = KeywordIndexSelector(
    keywords={
        "faq": ["beneficios", "vacaciones", "licencias"],
        "docs": ["proyecto", "reglamento", "informe"]
    },
    default="docs"
)

# Paso 2 - Definir Los retrievers
retrievers = {
    "faq": index_faq.as_retriever(),
    "docs": index_docs.as_retriever()
}

# Paso 3 - Crear el RouterRetriever
router = RouterRetriever(retrievers=retrievers, selector=selector)
```



También es posible construir un selector simple que no use modelos de lenguaje, sino que asocie consultas a retrievers usando reglas o palabras clave.

¿Cómo funciona KeywordIndexSelector?

- ★ Se define un conjunto de **palabras clave asociadas a cada índice**.
- ★ Cuando llega una consulta, el selector busca si alguna de esas palabras está presente.
- ★ Si hay coincidencia, se elige el retriever correspondiente.
- ★ Si no hay match, se usa un **retriever por defecto** (si lo definís).



ROUTERRETRIEVER CON LÓGICA ESTRUCTURADA (PYDANTICSELECTOR)

```
from llama_index.selectors import PydanticSelector
from pydantic import BaseModel

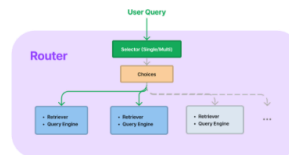
# Paso 1 - Definir el modelo de entrada
class Input(BaseModel):
    query: str

# Paso 2 - Crear función condicional
def is_project_query(data: Input) -> bool:
    return "proyecto" in data.query.lower()

# Paso 3 - Crear el selector
selector = PydanticSelector(
    choices=["docs", "faq"],
    conditions={
        "docs": is_project_query
    },
    default="faq"
)

# Paso 4 - Definir Los retrievers
retrievers = {
    "docs": index_docs.as_retriever(),
    "faq": index_faq.as_retriever()
}

# Paso 5 - Crear el RouterRetriever
router = RouterRetriever(retrievers=retrievers, selector=selector)
```



PydanticSelector permite definir reglas condicionales usando objetos estructurados, evaluando campos como la consulta del usuario o metadatos del contexto.

¿Para qué sirve?

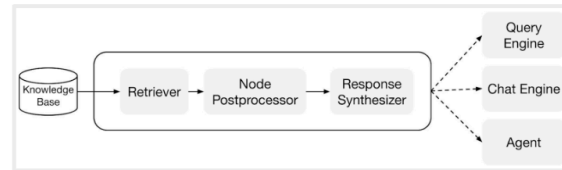
- ★ Para sistemas donde es posible anticipar estructuras o tipos de consulta.
- ★ También es muy útil en escenarios donde se requiere más control que un simple keyword match, pero sin LLM.





OTRAS HERRAMIENTAS DEL QUERY ENGINE: NODEPOSTPROCESSOR

NodePostprocessor permite filtrar, transformar o reordenar los nodos recuperados por el Retriever, antes de que lleguen al LLM para generar una respuesta.



¿Para qué sirve?

- ★ Filtrar nodos poco relevantes
- ★ Reordenar los resultados
- ★ Aplicar lógica condicional sobre los chunks

✦ También existen postprocesadores por metadatos (MetadataFilterPostprocessor) o por reordenamiento (Rerank con LLM o heurísticas).

```

from llama_index.postprocessor import (
    ScoreThresholdPostprocessor
)

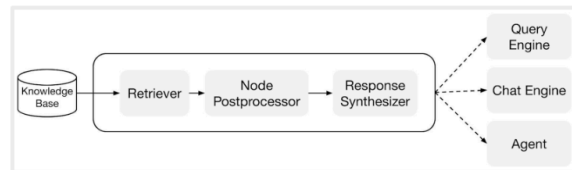
postprocessor = ScoreThresholdPostprocessor(
    threshold=0.75
)

query_engine = index.as_query_engine(
    similarity_top_k=5,
    node_postprocessors=[postprocessor]
)
  
```



OTRAS HERRAMIENTAS DEL QUERY ENGINE: RESPONSESYNTHESIZER

El componente ResponseSynthesizer se encarga de generar la respuesta final combinando la consulta del usuario con los nodos recuperados por el retriever (y opcionalmente filtrados por un postprocessor).



¿Para qué sirve?

- ★ Estructura la interacción entre el contexto y el LLM.
- ★ Define cómo se sintetizan los nodos para responder la consulta.
- ★ Permite elegir entre distintas estrategias de generación.

```

query_engine = index.as_query_engine(
    similarity_top_k=5,
    response_mode="refine"
)

response = query_engine.query("¿Qué dice el reglamento sobre licencias?")
  
```

Modos de respuesta disponibles

Modo	Estrategia de síntesis
compact	Concatena todos los nodos y genera una única respuesta
refine	Responde nodo por nodo, refinando progresivamente
tree_summarize	Crea resúmenes intermedios y los sintetiza jerárquicamente



REFERENCIAS BIBLIOGRÁFICAS

- ★ Documentación oficial de LlamaIndex <https://docs.llamaindex.ai>
- ★ Guía de introducción a RAG: <https://docs.llamaindex.ai/en/stable/understanding/rag>
- ★ Ejemplos prácticos (Query Engines, Tools, Agents, etc.): <https://docs.llamaindex.ai/en/stable/examples/>
- ★ Artículo base sobre RAG: <https://platform.openai.com/docs/guides/retrieval>
- ★ Repositorio oficial de LlamaIndex GitHub: https://github.com/jerryliu/llama_index
- ★ Question Answering in RAG using Llama-Index: Part 1 [\[URL\]](#)
- ★ Question Answering in RAG using Llama-Index: Part 2 [\[URL\]](#)

